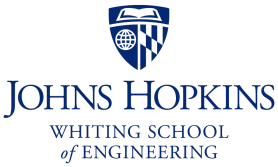
© Guven

Apriori Analysis



Introduction

> "A priori knowledge is independent of experience." Immanuel Kant

Apriori analysis and apriori algorithm for **frequent itemset mining** and **association rule learning** is used to find patterns in transactional datasets. An example is from online retailers where the page suggests the prospective buyer with the comment: "Who buys this also buys that". Today, online retailers must show customers the most relevant page since they carry millions of items for sale. This relevancy can be computed with apriori analysis. Another example is brick-and-mortar store product placement, such as placing related items close to each other so that the buyer does not have to walk much or the buyer buys it when he/she sees a related item. Famous beer-diaper story - Originated in 1992, according to an industry consultant, Teradata, when analyzing a retailer's database, discovered that there is a good chance when a customer buys diapers, the customer also buys beer. Of course, this discovered pattern was subtle but resulted in a statistically probable frequent itemset after the apriori analysis of the transactional dataset.

Association Rule Learning

A method to determine relations among variables in large databases is as follows:

Let $I = \{i_1, i_2, \ldots, i_m\}$ be a set of m features, representing distinct **items** (i.e., products).

Let $D = \{t_1, t_2, \ldots, t_n\}$ be the set of n **transactions** stored in a database D. Every t_i is a distinct event and consists of a subset of items from I.

A rule is an implication $X \rightarrow Y$ where X and Y are **itemsets** and subsets of I, and they have no element in common, i.e. $X \cap Y = \emptyset$

X is called **antecedent** and Y is called **consequent** of the rule.

An example could be {statistics book, algorithms book, programming book} $\rightarrow$ {machine learning book}

Clearly, extracting such relations from the datasets is a tedious task and naively has exponential time complexity. Given m items, there are 2^m possible candidate itemsets.

Support

The support of an itemset X, $\text{supp}(X)$ is the ratio of the number of transactions, which includes all the items of X to the total number of transactions. Naturally, it shows the popularity of the itemset, i.e. {CD recorder, CD blank media}

$$\text{supp}(X) = \frac{\text{number of} X \subseteq \text{transactions}}{\text{total number of transactions}} = \frac{|\{t \in D; X \subseteq t\}|}{|D|} \text{ ..... } (Eq.1)$$

For example, online retailers show item rankings to guide buyers. Computing all itemsets with one distinct product would give that ranking.

Confidence

Confidence (also corresponds to conditional probability) signifies the likelihood of item Y being purchased when item X is purchased.

$$\text{conf}(X \rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)} \text{ ..... } (Eq.2)$$

Lift

Lift shows the likelihood of itemset Y when X is purchased while normalizing with respect to the popularity of Y.

$$\text{lift}(X \rightarrow Y) = \frac{\text{supp}(X \cup Y)}{\text{supp}(X)\,\text{supp}(Y)} \text{ ..... } (Eq.3)$$

Notes on the Application and Algorithm

Considering transactional datasets where a buyer might have more than one transaction, the purchases of the same buyer can be combined, i.e., weekly or monthly, to make into itemsets, assuming the buyer purchases in a month have relevancy between the items purchased for a buyer.
Apriori algorithms such as Max-Miner, FP-growth use the following facts (apriori principle below):

- All subsets of a frequent itemset must be frequent
- Any infrequent itemset, all its supersets must be infrequent

Thus, applying a support threshold to prune the itemsets with lower support decreases the amount of data to be processed.

Generating frequent k-itemsets corresponds to finding all itemsets that have k items in them with support higher than a given `minsupport` threshold. Clearly, in order to generate efficiently the k-itemset, we compute (k-1)-itemset.

Apriori Principle

Anti-monotone property of support: Support of an itemset never exceeds the support of its subsets.

$$\forall X, Y : (X \subseteq Y) \implies \text{supp}(X) \geq \text{supp}(Y) \text{ ..... } (Eq.4)$$

Worked Example - Market Basket Analysis

Let's examine and run the code below on the store data file and process frequent itemsets that exist in the dataset. The dataset contains items purchased from a store, such as salad, turkey, tomatoes, etc. Each row is a list of items that were purchased in the same transaction by the customer.

The following cells will use a conversion, called **denormalization**, on the dataset list of transactions to another system, where each item presence is represented by a True or False, i.e., `0` or `1`. Note that the Weka framework also uses this representation.

Prospect Improvement

As seen by the enormous number of possible itemsets, it becomes infeasible to directly compute the support for k-itemsets, as k increases. Thus, a simple check has to be added to the support function, such that when a support of '$k <$' subset itemset is 0, the support for the k-itemset is returned as 0. This corresponds to the primitive application of $Eq.4$. This improvement still will not be enough to speed up computations, thus algorithms such as Max-Miner or FP-Growth are necessary.

In [1]:
```
from collections import defaultdict
import csv
from itertools import combinations
import numpy as np
```

```
Transactions_list = []  # a list of transactions
Items_names = {}  # Lookup item ID to name
Items_ids = {}  # Lookup item name to ID

Items = None  # a list of item IDs, normally an increasing sequence of num

# Process the data
with open('/EP_datasets/store_data.csv', 'r') as fin:
    reader = csv.reader(fin, delimiter=',')
    item_id = 0
    for row in reader:
        transaction = []
        for item in row:
            if item not in Items_ids:
                Items_ids[item] = item_id
                Items_names[item_id] = item
                item_id += 1
            transaction += [Items_ids[item]]
        Transactions_list += [transaction]

M, N = len(Items_ids), len(Transactions_list)

Items = np.arange(0,M)

# Information, sanity
print(f'M={M} items, N={N} transactions')
```

```
M=120 items, N=7501 transactions
```

In [2]:
```
# Sanity check
print([Items_names[_] for _ in Items[0:7]])
print(Transactions_list[:7])
```

```
['shrimp', 'almonds', 'avocado', 'vegetables mix', 'green grapes', 'whole we
at flour', 'yams']
[[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19], [2
0, 21, 22], [23], [24, 2], [14, 25, 26, 27, 11], [10], [28, 29]]
```

In [3]:
```
# Convert to numpy arrays
Transactions = np.full((N,M), False, dtype=bool)

for i, t in enumerate(Transactions_list):
    for item in t:
        Transactions[i][item] = True

# Sanity, print row index 10, 11
print(f'{Transactions[10:12].astype(int)}')
```

```
[[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1
  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0 0 0 0 0 0 0 0]]
```

In [4]:
```
# Based on equations
def support(_itemset:list)->int:
    global Transactions, N, M
    cnt = 0
    for transaction_i in range(N):  # all transactions
        bMatch = True
        for item_ in _itemset:
            if not Transactions[transaction_i][item_]:
                bMatch = False
                break
        if bMatch:
            cnt += 1
    return cnt

def confidence(_it1:list, _it2:list)->float:
    return support([_it1, _it2])/support([_it1])
```

In [5]:
```
Itemset_1, Itemset_2, Itemset_3 = {}, {}, {}
```

In [6]:
```
%%time

# 1-itemset supports
items_1 = list(combinations(Items, 1))
print(f'Size of possible 1-items= {len(items_1)}')

for itemset in items_1:
    Itemset_1[itemset] = support(itemset)

# Convert back to item names and sort
sorted_1 = list(map(lambda x: (tuple(Items_names[_] for _ in x[0]),x[1]),
sorted_1 = sorted(sorted_1, key=lambda kv: kv[1], reverse=True)

# Print top-2 and bottom-2
print('\n'.join(map(str, sorted_1[:3]+sorted_1[-2:])))
```

```
Size of possible 1-items= 120
(('mineral water',), 1788)
(('eggs',), 1348)
(('spaghetti',), 1306)
(('water spray',), 3)
((' asparagus',), 1)
CPU times: total: 125 ms
Wall time: 115 ms
```

In [7]:
```
%%time

# 2-itemset supports
items_2 = list(combinations(Items, 2))
print(f'Size of possible 2-items= {len(items_2)}')

for itemset in items_2:
    Itemset_2[itemset] = support(itemset)

# Convert back to item names and sort
```

```
sorted_2 = list(map(lambda x: (tuple(Items_names[_] for _ in x[0]),x[1]),
sorted_2 = sorted(sorted_2, key=lambda kv: kv[1], reverse=True)
```

```
Size of possible 2-items= 7140
CPU times: total: 6.59 s
Wall time: 6.59 s
```

In [8]:
```
# Find the first 0 support to mark the bottom
ix = next((i for i, x in enumerate([cnt for it,cnt in sorted_2]) if x==0),

# Print top-5 and bottom-2
print('\n'.join(map(str, sorted_2[:5]+sorted_2[ix-2:ix+1])))
```

```
(('mineral water', 'spaghetti'), 448)
(('mineral water', 'chocolate'), 395)
(('mineral water', 'eggs'), 382)
(('mineral water', 'milk'), 360)
(('mineral water', 'ground beef'), 307)
(('burger sauce', 'oatmeal'), 1)
(('oatmeal', 'napkins'), 1)
(('shrimp', 'cream'), 0)
```

In [9]:
```
%%time

# 3-itemset supports
items_3 = list(combinations(Items, 3))
print(f'Size of possible 3-items= {len(items_3)}')

for itemset in items_3:
    Itemset_3[itemset] = support(itemset)

# Convert back to item names and sort
sorted_3 = list(map(lambda x: (tuple(Items_names[_] for _ in x[0]),x[1]),
sorted_3 = sorted(sorted_3, key=lambda kv: kv[1], reverse=True)
```

```
Size of possible 3-items= 280840
CPU times: total: 4min 34s
Wall time: 4min 34s
```

In [10]:
```
# Find the first 0 support to mark the bottom
ix = next((i for i, x in enumerate([cnt for it,cnt in sorted_3]) if x==0),

# Print top-5 and bottom-2
print('\n'.join(map(str, sorted_3[:5]+sorted_3[ix-2:ix+1])))
```

```
(('mineral water', 'spaghetti', 'ground beef'), 128)
(('mineral water', 'spaghetti', 'chocolate'), 119)
(('mineral water', 'milk', 'spaghetti'), 118)
(('mineral water', 'eggs', 'spaghetti'), 107)
(('mineral water', 'milk', 'chocolate'), 105)
(('fromage blanc', 'melons', 'ketchup'), 1)
(('gluten free bar', 'cauliflower', 'bramble'), 1)
(('shrimp', 'almonds', 'chutney'), 0)
```

In [11]:
```
# 4-itemset supports
items_4 = list(combinations(Items, 4))
print(f'Size of possible 4-items= {len(items_4)}')
```

```
Size of possible 4-items= 8214570
```

Rules

A rule can be computed through the Confidence in $Eq.2$, which shows how often the rule is true in the dataset.

The confidence value of a rule,
$X \rightarrow Y$
is the proportion of the transactions that contain X, which also contains Y.

The following cells demonstrate rules extracted from 2-itemsets and 3-itemsets.

In [12]:
```
# Find high confidence item in 2-itemsets
Conf_2 = defaultdict(list)

for itemX in Items:
    for itemY in Itemset_2:
        if itemX in itemY:
            Conf_2[itemX] += [(itemY, Itemset_2[itemY]/Itemset_1[(itemX,)]
```

In [13]:
```
# Sort each item's confidence to find the best for each item
for item in Items:
    Conf_2[item] = sorted(Conf_2[item], key=lambda kv: kv[1], reverse=True

conf_top = np.array([Conf_2[i][0][1] for i in range(M)])

# Top confidence items for 2-itemsets
conf_top_top = conf_top.argsort()[-10:][::-1]
for ix in conf_top_top:
    print(f'{Conf_2[ix][0][1]:.3f} {Items_names[ix]:>15} -> {tuple(map(lam
```

```
1.000       asparagus -> ('shrimp', ' asparagus')
0.667     water spray -> ('shrimp', 'water spray')
0.571           cream -> ('spaghetti', 'cream')
0.487     nonfat milk -> ('mineral water', 'nonfat milk')
0.483   strong cheese -> ('spaghetti', 'strong cheese')
0.478      mayonnaise -> ('mineral water', 'mayonnaise')
0.478 extra dark chocolate -> ('mineral water', 'extra dark chocolate')
0.470     light cream -> ('mineral water', 'light cream')
0.457       asparagus -> ('mineral water', 'asparagus')
0.456            soup -> ('mineral water', 'soup')
```

In [14]:
```
# Find high confidence item in 3-itemsets
Conf_3 = defaultdict(list)

for itemX in Items:
    for itemY in Itemset_3:
        if itemX in itemY:
            Conf_3[itemX] += [(itemY, Itemset_3[itemY]/Itemset_1[(itemX,)]
```

In [15]:
```
# Sort each item's confidence to find the best for each item
for item in Items:
```

```
    Conf_3[item] = sorted(Conf_3[item], key=lambda kv: kv[1], reverse=True

conf_top = np.array([Conf_3[i][0][1] for i in range(M)])

# Top confidence items for 3-itemsets
conf_top_top = conf_top.argsort()[-10:][::-1]
for ix in conf_top_top:
    print(f'{Conf_3[ix][0][1]:.3f} {Items_names[ix]:>15} -> {tuple(map(lam
```

```
1.000       asparagus -> ('shrimp', 'burgers', ' asparagus')
0.429           cream -> ('mineral water', 'spaghetti', 'cream')
0.400         napkins -> ('low fat yogurt', 'spaghetti', 'napkins')
0.333     water spray -> ('shrimp', 'frozen vegetables', 'water spray')
0.241   strong cheese -> ('mineral water', 'spaghetti', 'strong cheese')
0.231     nonfat milk -> ('mineral water', 'spaghetti', 'nonfat milk')
0.227    burger sauce -> ('mineral water', 'spaghetti', 'burger sauce')
0.226         chutney -> ('mineral water', 'eggs', 'chutney')
0.217    tomato sauce -> ('spaghetti', 'ground beef', 'tomato sauce')
0.214         bramble -> ('green tea', 'turkey', 'bramble')
```

The discovered first rule says that if the item 'asparagus' is purchased, then there is a 100% chance that ('shrimp', 'burgers') are also purchased. Since support or the probability of an itemset in the dataset drives most of this derivation, Apriori analysis works best on huge amounts of transactions, otherwise discovered rules might be just misleading. For example, the support of 'asparagus' is very low, i.e., 1 count, and whenever it shows up, the rest of the items ('shrimp', 'burgers') will be purchased since it is the only transaction we have with 'asparagus'. Thus, the rule is **correct** but **useless** (e.g., it can be considered as an anomaly).

As the above cells show, fast algorithms using the Apriori principle are mandatory to work with large datasets.

Question: Did you notice support is diminishing as we add more items to the itemsets?

References

1. Agrawal, Rakesh, Tomasz Imielinski, and Arun Swami. "Mining association rules between sets of items in large databases." Proceedings of the 1993 ACM SIGMOD international conference on Management of data. 1993.
2. Witten, I. H., et al. "Practical Machine Learning." (2017)

Exercises

Exercise 1. The market dataset can use a simple NLP cleaning to improve the words, such as removing leading spaces, etc. Attempt to improve the above results.

Market Basket Analysis with `mlxtend` Library

Install with `conda install conda-forge::mlxtend`

mlxtend (machine learning extensions) is a Python library of useful tools for the day-to-day data science tasks. (ref https://rasbt.github.io/mlxtend/)

In [16]:
```
import pandas as pd
import mlxtend
from mlxtend.frequent_patterns import apriori, association_rules

print(f'mlxtend version= {mlxtend.__version__}')

# Create one hot DataFrame
df = pd.DataFrame(Transactions, columns=Items_names.values())

df_apriori = apriori(df, min_support=0.001, use_colnames=True, max_len=3)
print(f'{len(df_apriori)} many itemsets')
df_apriori.tail()
```

```
mlxtend version= 0.22.0
5612 many itemsets
```

Out[16]:

	support	itemsets
5607	0.001200	(grated cheese, ground beef, butter)
5608	0.001067	(herb & pepper, fresh bread, ground beef)
5609	0.001600	(herb & pepper, escalope, ground beef)
5610	0.001200	(herb & pepper, ground beef, tomato sauce)
5611	0.001333	(herb & pepper, ground beef, cake)

In [17]:
```
df_rules = association_rules(df_apriori, metric='confidence', min_threshol
print(f'{len(df_rules)} many rules')
df_rules.head()
```

```
5094 many rules
```

Out[17]:

	antecedents	consequents	antecedent support	consequent support	support	confidence	li
0	(shrimp)	(mineral water)	0.071457	0.238368	0.023597	0.330224	1.38535
1	(shrimp)	(milk)	0.071457	0.129583	0.017598	0.246269	1.90047
2	(shrimp)	(frozen vegetables)	0.071457	0.095321	0.016664	0.233209	2.44657
3	(shrimp)	(spaghetti)	0.071457	0.174110	0.021197	0.296642	1.70376
4	(shrimp)	(chocolate)	0.071457	0.163845	0.017998	0.251866	1.53722

Market Basket Analysis with Weka Framework

Weka framework has an association rules analytic, which includes the fast FP-Growth algorithm.

A denormalization is required to load the `store_data.csv` store dataset to Weka. The below function creates a CSV file where each transaction is mapped to a row of binary values for every distinct item i_m

---

WATCH THE RELATED VIDEO for Apriori analysis using the Weka framework.

In [18]:
```
def save_file(_fn):
    global Transactions, Items_names, N, M
    with open(_fn, 'w') as fout:
        writer = csv.writer(fout, delimiter=',', quoting=csv.QUOTE_ALL, qu
        writer.writerow([Items_names[i] for i in range(M)])
        for i in range(N):
            writer.writerow(list(map(lambda x: '' if x == False else 'True

save_file('input_for_weka.csv')
```

In [19]:
```
%%html
<style>
    table {margin-left: 0 !important;}
    p {font-family: verdana;}
    li {font-family: verdana;}
    div {font-size: 10pt;}
</style>
<!-- Display markdown tables left oriented in this notebook. -->
```
